프로젝트 설명서

1. 프로토타입 제목			
프로젝트명	대학생활 통합 지원 시스템	팀명	졸업시켜조
팀원	박시현(202432734), 백승혜(202434982), 이종혁(202235305), 정회훈(202235330)		
2. 프로토타입 설명			
What	대학 졸업에 있어서 필요한 학점, 스펙, 구직 활동의 관리를 보조		
Who	체계적인 목표 달성과 효율적인 시간 관리를 원하는 모든 대학생		
When	재학생의 수강 관리부터 졸업반의 구직 활동까지의 대학 생활 전반		
Why	학업과 취업 준비에 대한 가이드를 제공받기 위해		

3. 업무 분장 표			
역할별 분담		기능별 분담	
박시현	전체 문서화, 총괄	박시현	스터디 플래너(학업/성적 관련 기능)
백승혜	기능 통합	백승혜	비교과 추천 & 스펙 성장 로드맵(비교과 관련 기능)
이종혁	기능 통합	이종혁	취업 합격률 예측 서비스(취창업 관련 기능)
정회훈	ppt, 발표	정회훈	졸업요건 진단 시스템(졸업요건/학점 진단 기능)

사용한 자료구조/알고리즘		
박시현	큐, 해시 테이블(체이닝)	병합 정렬, 그리디 알고리즘
백승혜	우선순위 큐	점수 기반 추천 알고리즘, Graph(경로 탐색) 개념
이종혁	우선순위 큐, 트리, 해시 집합	DFS, 이진 탐색, 그리디 알고리즘
정회훈	해시 맵, 선형 리스트, 해시 집합	선택 정렬, 그리디 알고리즘

이하는 각 기능에 대한 자세한 설명이며, 다음 내용을 포함함.

4. Input 데이터

5. 사용한 자료구조 & 알고리즘

6. 설계/구현 방법

7. 최종 완성 결과 캡처

8. Lessons learned

1. 스터디 플래너 - 202432734 박시현

기능명	스터디 플래너	담당자	202432734 박시현
1-4. Input 데이터			
과목명, 과목 학점, 시험 디데이, 전공 여부 등을 '사용자가 직접' 입력한다. (외부 출처 x) - 고유한 과목명을 Key로, 세부 속성(학점, 전공 여부...)들을 딕셔너리로 묶어 Value로 삼는 Key-Value 페어 구조			

- 내부적으로는 체이닝 방식 해시 테이블의 각 버킷 내에 연결 리스트 형태로 관리.	
1-5. 사용한 자료구조 & 알고리즘	
자료구조	
큐 (Queue)	CustomQueue 클래스로 직접 구현됨 - 위치: calculate_schedule 함수 내의 최종 스케줄 저장(study_queue) 스케줄링이 완료된 '학습 계획'을 담아두는 용도. - 선택 이유: 산출된 학습 계획은 순서대로 하나씩 실행하고 완료해야 하기에 먼저 스케줄링된 과목을 먼저 처리하는 FIFO(First-In-First-Out) 구조가 적절하다고 판단
해시 테이블 (Hash Table)	CustomHashTable 클래스로 직접 구현됨 - 위치: 전역 객체 subjects의 생성/등록/조회/삭제 작업 전반 - 선택 이유: 스케줄러 특성상 특정 과목을 찾아 삭제하거나 조회하는 작업이 빈번하기 때문에 과목명을 인덱스로 하여 평균 O(1)의 시간 복잡도로 데이터에 즉시 접근할 수 있도록 함. 충돌은 같은 버킷에 연결 리스트로 저장하여 해결하는 체이닝 기법 사용.
알고리즘	
병합 정렬 (Merge Sort)	- 위치: merge_sort 메서드 - 선택 이유: 과목별 우선순위 점수(priority_score)를 기준으로 데이터를 내림차순 정렬할 때 사용하며, 구현이 쉬우면서도 항상 O(N log N)의 시간 복잡도를 보장하여 최악의 경우에도 안정적인 성능을 낸다는 점과 안정 정렬이기 때문에 점수가 같은 과목 간의 순서가 뒤섞이지 않는다는 점을 고려하여 선택.
그리디 알고리즘 (Greedy Algorithm)	- 위치: calculate_planner_schedule 메서드 내의 공부 시간 분배 (allocated_time) - 선택 이유: 제한된 공부 시간 안에서 당장 가장 급한(우선순위가 높은) 과목에 최대한 많은 시간을 할당하기 위함이며, 동적 가중치 조절을 통해 밸런스를 유지함.
1-6. 설계/구현 방법	
Architecture	CustomHashTable ↓ items() merge_sort() ↓ 정렬된 리스트 그리디 알고리즘 ↓ 배분 결과 CustomQueue ↓ dequeue 화면 출력
동작 방식	1. 사용자가 과목 정보 입력 후 과목 추가 버튼을 누르면 예외 처리(빈 과목명, 과거 날짜 등)로 데이터 검증 후 해시 테이블(CustomHashTable)에 상태 값(노력도, 학점, D-day 등)을 삽입 2. 계획 생성 버튼을 누르면 각 과목의 '(노력도 * 학점) / D-day * 전공가중치

	(1.5 또는 1.0)' 공식을 통해 우선순위 점수를 산출하여 해시 테이블을 업데이트 3. 해시 테이블의 데이터를 리스트로 추출하여 병합 정렬(merge_sort())을 통해 우선순위 점수를 기준으로 내림차순 정렬 4. 총 가용 시간을 0.1시간 단위로 쪼개고, 그리디 알고리즘을 적용하여 현재 점수가 가장 높은 과목에 시간을 할당. 이때 한 과목의 시간 독점을 방지하기 위해, 전체 가용 시간 대비 단계별 비율(1.0 / total_steps)만큼 해당 과목의 점수를 차감하여 전체 가용 시간 동안 점수가 균등하게 소진되도록 유도 5. 시간 배분이 완료된 스케줄을 큐(CustomQueue)에 순차적으로 삽입하고, 다시 큐에서 차례대로 추출하여 최종 학습 순서 가이드 문자열로 조합 및 반환
핵심 코드	1) 우선순위 산출 로직 2) 그리디 배분 로직

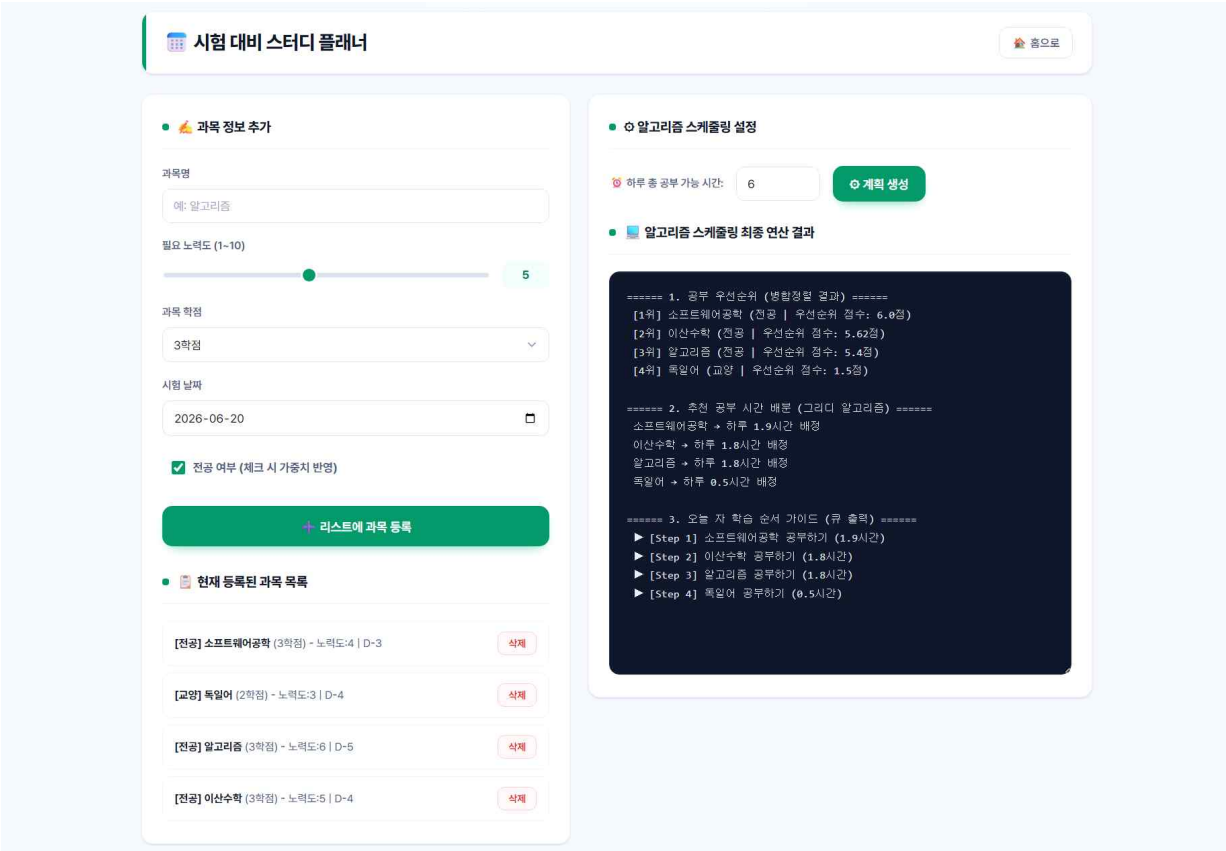
1) 우선순위 산출 로직

```python
    for name, info in planner_subjects.items():
        base_score = (info["effort"] * info["credits"]) / info["d_day"]
        info["priority_score"] = base_score * (1.5 if info["is_major"] else 1.0)
        planner_subjects.set(name, info)
```

2) 그리디 배분 로직

```python
        while remaining_time >= time_unit and any(score > 0 for score in current_priorities.values()):
            best_subject = max(current_priorities, key=current_priorities.get)

            if current_priorities[best_subject] <= 0:
                break

            allocated_times[best_subject] += time_unit
            remaining_time -= time_unit

            current_priorities[best_subject] -= (total_priority * dynamic_penalty_rate)
```

1-7. 최종 완성 결과 캡처



1-8. Lessons learned	
Plus	- 팀 프로젝트에서 필요한 역량들을 보완하는 계기 - 사용한 자료구조와 알고리즘의 활용 목적을 구체적으로 이해 - 실생활 서비스 개발을 통해 알고리즘의 실용성을 체감 - 팀 활동을 통해 협업의 중요성을 체감
Minus	- 기능에 대한 최선의 알고리즘을 선택하지 못함 - 사용자의 입력이 많아 번거로울 수 있음 - 팀원 간 일정 조율의 어려움 - 팀원마다 개발 속도와 구현 방식이 달라 코드 통합 과정에서 예상보다 많은 시간이 소요
Interesting	- tkinter GUI 기반 프로그램을 처음 작성해 봄 (이후 기능 통합으로 삭제) - 다양한 관점을 가진 팀원들과 협력하는 개발 과정 경험 - 다른 사람의 코드를 분석, 피드백 해보는 경험

2. 비교과 추천 & 스펙 성장 로드맵 - 202434982 백승혜

기능명	비교과 추천 & 스펙 성장 로드맵	담당자	202434982 백승혜
2-4. Input 데이터			
EXTRA_PROGRAMS (15개) - 형태: List of Lists ([[프로그램명, 분야, 타겟직무, 최소학년, 최소학점], ...]) - 비교과 프로그램의 지원 자격(학년/학점)과 연관 카테고리 정보를 담고 있는 핵심 마스터 데이터이다. EXTRA_ROADMAP (6개 직무) - 형태: Key-Value Dictionary ({ 직무명: [단계별 프로그램 리스트] }) - 직무별로 권장하는 커리어 로드맵의 순차적 단계를 정의한다. - EXTRA_LINKS & EXTRA_FIELD_SITES - 형태: Dictionary 및 Dictionary of Tuples - 프로그램별 상세 페이지 URL 및 분야별 추천 외부 사이트 링크 정보를 매핑한다.			
2-5. 사용한 자료구조 & 알고리즘			
자료구조			
우선순위 큐 (Priority Queue)	- 사용된 위치코드 라인: pq = [] 객체 생성 및 최종 while pq: 루프를 통한 데이터 추출 구간 - 역할: 사용자 맞춤형 점수가 가장 높은 추천 프로그램을 가장 먼저, 순서대로 꺼내오는 '대기열' 역할을 한다. - 선택 이유: 일반적인 큐는 먼저 들어간 데이터가 먼저 나오는 구조지만 추천 시스템에서는 등록된 순서가 아니라, "사용자에게 얼마나 적합한가(점수가 높은가)"가 기준이 되어야 한다. 또한 추천 데이터가 실시간으로 가중치 계산을 거쳐 나올 때마다 매번 리스트 전체를 정렬(sort())하는 것은 비효율적이다. 데이터가 들어올 때마다 자신이 위치해야 할 우선순위를 알아서 찾아가도록 돕는 추상적 개념이 필요했으며, 이에 가장 부합하는 정렬된 대기열 모델이 '우선순위		

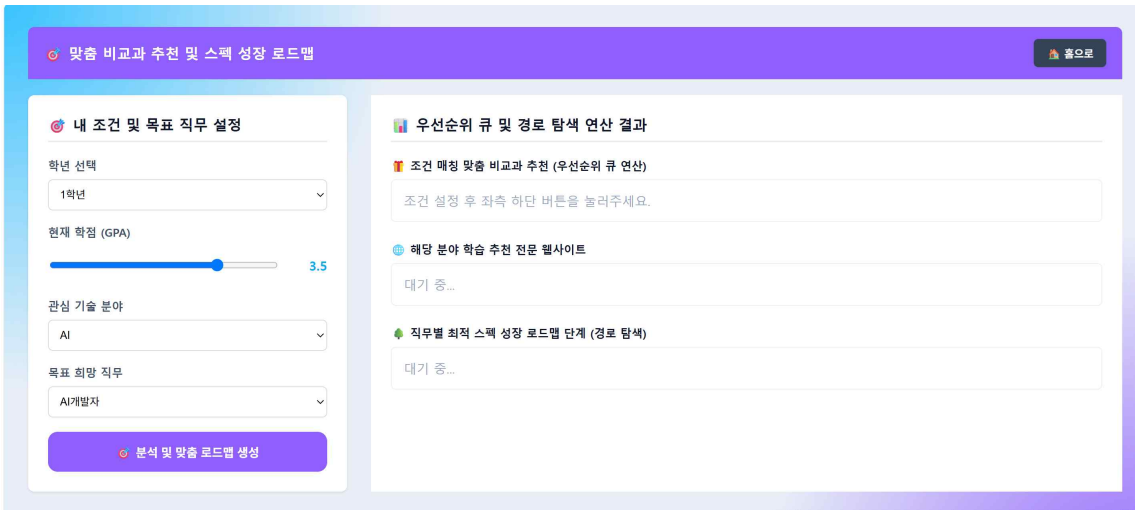
	큐'라고 생각했다.
알고리즘	
점수 기반 추천 알고리즘	- 위치 : score += 50 - 용도 : 분야, 직무 일치도 계산 - 사용 이유 : 사용자 관심도와 직무 적합도를 수치화하여 개인 맞춤 추천을 제공한다.
Graph(경로 탐색) 개념	- 위치 : roadmap_service() - 용도 : 직무 성장 단계를 순서대로 제공한다. - 사용 이유 : 단순 목록이 아닌 경로(단계) 형태의 성장 흐름 표현이 가능하여 개인 맞춤 추천을 제공한다.
힙 (Heap) 알고리즘	- 사용된 위치코드 라인: heapq.heappush(pq, (-score, p_name)), heapq.heappop(pq) - 역할: 우선순위 큐라는 개념을 메모리 상에서 실제로 정렬하고 유지하는 내부 핵심 알고리즘이다. - 선택 이유: (O(log N)):N개의 데이터 중 가장 점수가 높은 항목을 찾을 때, 일반 리스트에서 순차 탐색을 하면 O(N)이 걸리고, 전체 정렬을 하면 O(N log N)이 소요된다. 반면 이진 트리 구조를 기반으로 한 힙 알고리즘을 사용하면 최악의 상황에서도 삽입(heappush)과 삭제(heappop) 시 O(log N)의 효율적인 성능을 보장한다. Python의 heapq 모듈은 기본적으로 가장 작은 값이 위로 올라오는 '최소 힙(Min-Heap)'으로만 작동한다. 점수가 높은 프로그램을 먼저 추천해야 하는 본 서비스의 목적에 맞추기 위해, 점수에 음수 부호(-score)를 붙여서 힙에 밀어 넣는 방식을 취했다. 가장 큰 양수 점수가 가장 작은 음수 점수가 되어 트리의 최상단(Root)에 위치하게 되는 최대 힙 효과를 완벽하게 구현할 수 있기 때문에 선택했습니다.
2-6. 설계/구현 방법	
Architecture	Flask 웹 서버 (/api/extra/analyze) ⬇ JSON 데이터 수신 (grade, gpa, field, job) ⬇ 조건 필터링 & 가중치 스코어링 (지원 자격 및 매칭 점수 계산) ⬇ 최대 힙 (heapq) 알고리즘 (부호 반전 기법 -score 적용) ⬇ 우선순위 큐 (Priority Queue) 생성 (점수 기준 내림차순 정렬) ⬇ 해시 맵 (Dictionary) 매핑 (직무 로드맵 및 웹 사이트 링크 조회) ⬇ JSON 응답 반환 및 화면 출력
동작 방식	1. 클라이언트가 사용자의 학년, 학점, 관심 분야, 희망 직무 정보를 담아 POST 요청을 보내면, 서버는 try-except 블록을 통해 입력 데이터의 유효성(형변환 가능

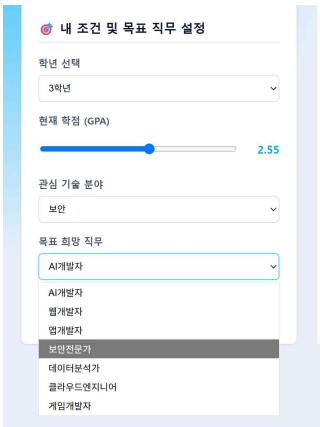
	여부)을 검증하고 예외를 처리한다. 2. 마스터 데이터인 EXTRA_PROGRAMS 배열을 순회하며 사용자의 학년과 학점이 최소 지원 자격(grade >= min_grade 및 gpa >= min_gpa)을 충족하는지 1차 필터링을 수행한다. 3. 자격을 충족한 프로그램에 한해 사용자의 관심 분야와 희망 직무가 매칭되는지 확인합니다. 일치하는 항목마다 각각 가중치 점수(+50점)를 부여하고, 공통 프로그램일 경우 추가 점수(+20점)를 합산하여 최종 우선순위 점수를 산출한다. 4. Python의 heapq 모듈을 활용하여 데이터의 삽입과 정렬을 최적화합니다. 이때 모듈의 최소 힙(Min-Heap) 특성을 극대화하기 위해 점수에 음수 부호(-score)를 붙여 밀어 넣음으로써, 가중치가 가장 높은 프로그램이 큐의 최상단에 위치하도록 최대 힙(Max-Heap) 구조를 구현한다. 5. 우선순위 큐에서 데이터를 차례대로 추출하면서 양수 점수로 재변환하고, 해시 맵 구조의 데이터(EXTRA_LINKS)에서 각 프로그램의 상세 URL을 $O(1)$의 속도로 매핑하여 최종 추천 리스트를 정제한다. 6. 사용자가 선택한 희망 직무(job)에 해당하는 커리어 단계별 로드맵과 관심 분야별 팁 사이트 데이터를 딕셔너리 구조에서 즉시 조회하여 결합한 뒤, 최종 완성된 데이터셋을 JSON 형태로 클라이언트에 반환한다.

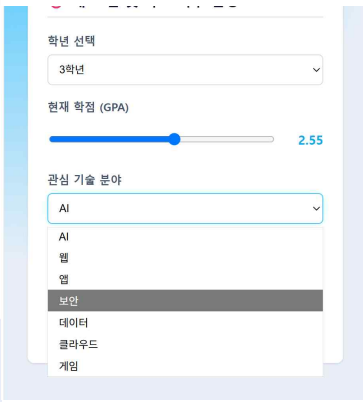
핵심 코드	알고리즘명: 선형 탐색 (Linear Search)
	for row in EXTRA_PROGRAMS: # O(N) 선형 탐색 시작
	  p_name, p_field, p_job, min_grade, min_gpa = row
	  score = 0
	
	  조건 검사 및 매칭 (선형 탐색의 핵심 조건문 분기)
	  if grade >= min_grade and gpa >= min_gpa:
	  if field == p_field: score += 50
	  if job == p_job: score += 50
	  if p_field == "공통": score += 20
	
	자료구조: 우선순위 큐 (Priority Queue) / 알고리즘: 힙 삽입 (Heap Push)
	역할: 점수에 음수(-) 부호를 붙여 최소 힙 유틸리티를 최대 힙 정렬 구조로 전환하여 데이터 삽입
	if score > 0 or p_field == "공통":
	  # heapq.heappush를 통해 이진 트리 구조 내에서 O(log N) 속도로 우선순위 정렬 유지
	  heapq.heappush(pq, (-score, p_name))
	
	자료구조명: 해시 테이블 / 맵 (Hash Table / Map - Python Dictionary)
	역할: 프로그램명(Key)과 직무명(Key)을 기반으로 매핑된 데이터를 O(1) 속도로 즉시 탐색
	1) 프로그램명을 Key로 하여 상세 URL 조회
	link = EXTRA_LINKS.get(name, "#")
	2) 직무명을 Key로 하여 딕셔너리에 정의된 단계별 로드맵 배열 조회

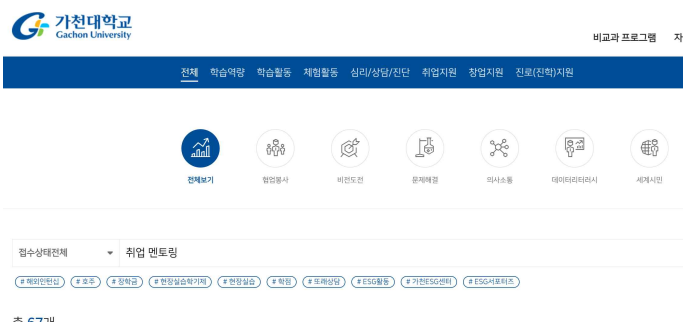
```
if job in EXTRA_ROADMAP:
    for idx, item in enumerate(EXTRA_ROADMAP[job]):
        link = EXTRA_LINKS.get(item, "#")
```

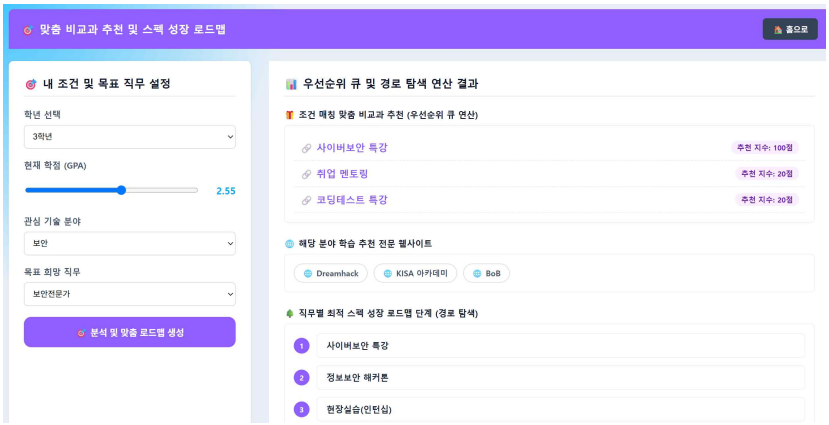
2-7. 최종 완성 결과 캡처











2-8. Lessons learned	
Plus	- 이론과 실무의 결합: 자료구조 수업에서 텍스트로만 접했던 '우선순위 큐'와 '힙(Heap)'의 개념을 사용자의 조건에 따른 '추천 시스템 정렬 알고리즘'이라는 실제 서비스 기능에 성공적으로 적용했다. 단순히 정적 리스트를 보여주는 방식을 넘어, 조건에 맞는 데이터를 동적으로 필터링하고 가중치를 매기는 백엔드 로직을 직접 설계해 보며 개발 역량이 한 단계 성장함을 느꼈다. - 시간 복잡도 최적화: 가중치 점수가 계산될 때마다 전체 배열을 매번 재정렬하는 비효율적인 방식 대신, 삽입과 추출 시 O(\log N)의 효율성을 보장하는 `heapq`를 사용하여 서버의 자원 소모를 줄이고 응답 속도를 최적화한 점이 매우 만족스럽다. - 사용자 중심의 가중치 설계: 단순 매칭이 아니라 관심 분야와 희망 직무에 각각 `+50점`을 부여하고, 자칫 소외될 수 있는 '공통 역량 프로그램'에도 `+20점`을 부여하여 정교하고 밸런스 있는 추천 결과를 도출해 낸 점이 구조적으로 훌륭했다고 생각한다. - 팀원들과의 협업 시너지: 팀원의 요구사항과 피드백을 수렴하여, 동적 가중치 추천 로직을 설계함으로써 팀원 간의 아이디어를 코드로 완벽히 구현해 냈습니다.
Minus	- 데이터 관리의 정적 구조 (하드코딩): 현재 비교과 마스터 데이터(`EXTRA_PROGRAMS`)와 링크 정보가 백엔드 소스 코드 내에 배열과 딕셔너리 형태로 하드코딩되어 있다. 이로 인해 새로운 프로그램이 개설되거나 기존 가천대 WIND 시스템의 URL이 변경될 경우, 관리자가 DB나 관리자 페이지가 아닌 '서버 소스 코드 자체를 수정하고 재배포'해야 하는 치명적인 운영상의 한계를 인지했습니다. 추후 데이터베이스(DB) 연동을 통해 동적으로 데이터를 관리하도록 개선이 필요하다. - 정교하지 못한 예외 처리와 동점자 처리: 사용자가 문자열을 잘못 입력하거나 빈 값을 보냈을 때 `ValueError`에 대한 기본적인 방어 코드는 작성했으나, 구체적으로 어떤 필드가 잘못되었는지 상세한 에러 메시지를 프론트엔드에 전달하지 못해 아쉽다. 또한, 두 개 이상의 프로그램이 동일한 최고 점수(예: 100점)를 가졌을 때, 2차 정렬 기준(예: 학년 제한이 낮은 순, 혹은 학점 기준이 높은 순)이 없어 정렬 순서가 힙의 내부 구조에 의존해 무작위로 나오는 현상을 보완해야 한다.
Interesting	- 언어적 한계를 극복한 트릭: Python의 `heapq` 라이브러리가 오직 '최소 힙(Min-Heap)' 기능만 지원한다는 사실을 알고 처음에 당황했다. 하지만 점수에

	음수 부호(`-score`)를 붙여 저장하면 가장 큰 양수가 가장 작은 음수가 되어 트리 최상단에 올라오게 되는 '부호 반전 트릭'을 활용하면서, 언어나 라이브러리의 제약 조건을 논리적 아이디어로 극복하는 개발의 묘미를 배웠다. - 네트워크 환경에 대한 이해: 로컬에서 개발할 때는 잘 뜨던 서버가 외부 연동 테스트 중 `ERR_NGROK_8012 (Connection Refused)` 에러를 뿜으며 일시적으로 먹통이 되는 현상을 겪었다. 이 과정에서 Flask의 `host='0.0.0.0'` 설정이 가진 의미와 IPv4(`127.0.0.1`) 및 IPv6(`[::1]`) 간의 호스트 바인딩 차이, 그리고 ngrok과 같은 터널링 프로그램이 로컬 포트와 어떻게 상호작용하는지 네트워크 인프라 측면의 지식을 흥미롭게 체감할 수 있었다.

3. 취업 합격률 제공 서비스 - 202235305 이종혁

기능명	취업 합격률 제공 서비스	담당자	202235305 이종혁
3-4. Input 데이터			
Input 데이터(출처: 잡코리아, 원티드) :accepted_spec.csv company_requirements.csv			
3-5. 사용한 자료구조 & 알고리즘			
자료구조			
해시 맵 (Hash Map)	-위치 : data, row_data, DataFrame 컬럼 접근 -사용 이유 : 사용자 입력값과 기업 데이터를 Key-Value 형태로 저장하여 필요한 값을 빠르게 가져오기 위해 사용하였다. 실제 서비스화된다고 가정했을 때에는 데이터의 양이 상당히 방대해질 수 있는데 기업명, 직무, 학점 등의 데이터를 효율적으로 조회할 수 있어 처리 속도를 높일 수 있다.		
해시 집합 (Hash Set)	-위치 : chosen_skills -사용 이유 : 사용자의 기술 스택을 중복 없이 저장하고, 특정 기술의 보유 여부를 빠르게 확인하기 위해 사용하였다. 리스트보다 탐색 속도가 빨라 기술 스택 비교에 효율적이다.		
순차 자료구조 / 배열 (Array / List)	- 위치 : all_companies, res_table, clean_table - 사용 이유 : 기업 목록과 합격률 결과 데이터를 순서대로 저장하고 관리하기 위해 사용하였다. 반복문 처리와 정렬 기능 사용이 편리하여 결과 출력에 적합하다.		
일반 트리 (General Tree)	-위치 : TreeNode 클래스 -사용 이유 : 기업 맞춤형 로드맵을 계층 구조로 표현하기 위해 사용하였다. 상위 항목과 하위 항목의 관계를 구조적으로 표현할 수 있어 사용자에게 시각적으로 보기 쉬운 피드백을 제공할 수 있다.		
알고리즘			
해시 탐색 (Hash	위치 : if s in chosen_skills -사용 이유 : 기업이 요구하는 기술 스택이 사용자의 보유 기술 목록에 포함되어		

Search)	있는지 빠르게 확인하기 위해 사용하였다. Set 자료구조를 활용하여 효율적인 탐색이 가능하다.
단순 비교 및 휴리스틱 (Heuristic)	- 위치 : 점수 계산 수식 분기문 (if/else 및 min/max) - 역할 및 매커니즘 : [규칙 기반 합격률 추정] 단순 대소 비교를 통해 사용자의 학점·어학 점수와 합격자 평균을 매칭하고, 가감산 및 가중치 수식(도메인 지식 규칙)을 적용해 실시간으로 점수를 판정하는 알고리즘.
팀소트 (Timsort)	- 위치 : res_table.sort(...) - 사용 이유 : 계산된 기업별 합격률 결과를 높은 순서대로 빠르게 정렬하기 위해 사용하였다. 일반적으로는 애초에 데이터 자체가 정렬되어 있을 가능성이 높으므로 팀소트가 제일 효율적인 정렬 알고리즘일거라고 판단했다.
깊이 우선 탐색 (DFS) + 재귀 알고리즘	- 위치 : render_roadmap_tree() 재귀 함수 - 사용 이유 : 로드맵 데이터를 트리 형태로 저장했기 때문에, 부모 노드부터 자식 노드 끝까지 순서대로 탐색하며 출력하기 위해 사용하였다. 재귀 함수를 이용하면 계층 구조를 자연스럽게 표현할 수 있고, 구현이 비교적 간단하여 트리 전체를 효율적으로 순회할 수 있다.
3-6. 설계/구현 방법	
Architecture	사용자가 입력한 학점, 어학 점수, 자격증, 인턴 경험, 활동 내역 및 보유 기술 스택을 기반으로 실제 합격자들의 평균 스펙과 기업의 요구사항과 비교하여 기업별 합격 가능성을 분석한다. 또한 분석 결과를 바탕으로 사용자의 부족한 역량을 시각적으로 확인할 수 있도록 트리(Tree) 구조 기반 로드맵을 생성한다. 데이터 저장은 CSV 기반으로 구현하였으며, 서버 실행 시 Pandas DataFrame 형태로 메모리에 적재하여 빠른 탐색이 가능하도록 설계하였다
동작 방식	1.시스템은 accepted_spec.csv와 company_requirements.csv 파일을 로드하여 기업별 평균 스펙 데이터와 요구 기술 스택 데이터를 메모리에 저장한다. 2.사용자가 입력한 스펙 정보는 JSON 형태로 전달되며, 서버는 이를 수신하여 정수형 및 실수형으로 안전하게 변환한다. 3.본 시스템의 핵심 알고리즘은 휴리스틱(Heuristic) 기반 합격률 분석 알고리즘이다. 기업별 평균 스펙과 사용자의 스펙을 비교하여 다음 요소를 점수화하였다. 학점, 어학 성적, 자격증, 인턴 경험, 대외활동, 기술 스택 일치율. 특히 기술 스택 비교는 Python의 Set 자료구조를 활용하여 평균 O(1)의 탐색 성능을 확보하였다.

핵심 코드	본 시스템의 핵심 기능은 사용자의 스펙과 기업 평균 요구 역량을 비교하여 합격 가능성을 예측하는 휴리스틱(Heuristic) 기반 알고리즘이다.
	특히 학점, 어학 성적, 기술 스택, 자격증, 인턴 경험 등을 종합적으로 반영하여 현실적인 합격률을 계산하도록 설계하였다.
	1. 기술 스택 일치율 계산
	skill_score = 0.0
	for s in req_skills:
	  if s in chosen_skills:
	  skill_score += 5.0

skill_score = min(20.0, skill_score)
기업이 요구하는 기술 스택(req_skills)과 사용자가 보유한 기술 스택(chosen_skills)을 비교하여 점수를 계산한다.
이 과정에서 Python의 Set 자료구조를 활용하여 평균 탐색 시간복잡도: O(1)
수준의 빠른 탐색 성능을 확보하였다.
2. 학점 및 어학 점수 계산

```
if gpa >= target_gpa:
    gpa_score = 20.0
else:
    gpa_score = max(
        0.0,
        20.0 - (target_gpa - gpa) * 20
    )
if english >= target_eng:
    eng_score = 10.0
else:
    eng_score = max(
        0.0,
        10.0 - (target_eng - english) * 0.05
    )
```

사용자의 학점 및 어학 성적을 기업 평균 데이터와 비교하여 점수를 계산한다.
기업 평균 이상일 경우 최대 점수를 부여하며, 평균 이하일 경우 차이에 비례하여 점수를 감점하는 방식으로 구현하였다.
이를 통해 실제 채용 과정에서의 정량 평가 방식을 반영하였다.
3. 추가 활동 점수 계산

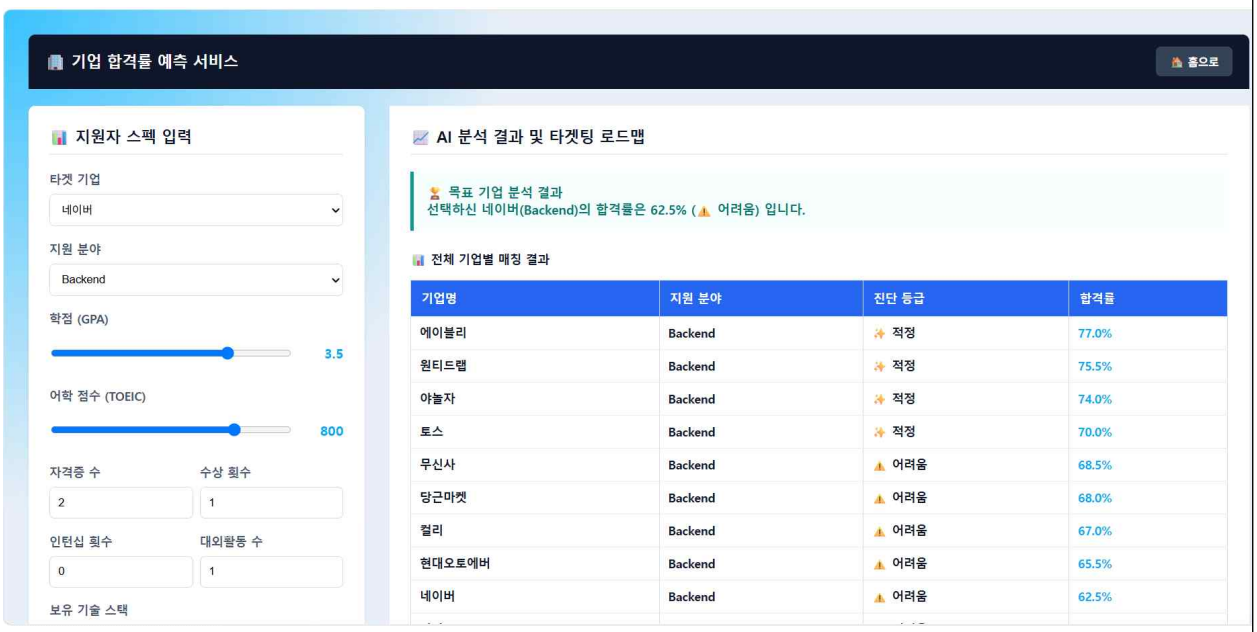
```
extra_score = min(
    10.0,
    (cert * 1.5) +
    (award * 2.0) +
    (intern * 4.0) +
    (act * 1.0)
)
```

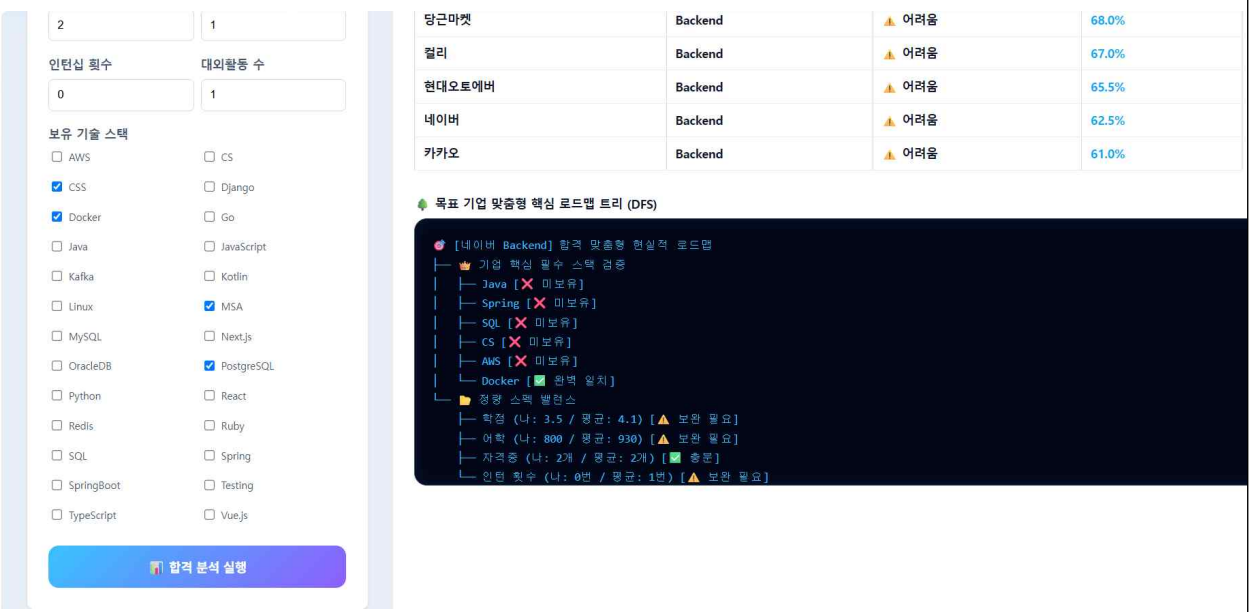
자격증, 수상 경력, 인턴 경험, 대외활동 등을 종합적으로 평가하여 추가 점수를 계산한다.
특히 인턴 경험에 높은 가중치를 부여하여 실무 경험의 중요성을 반영하였다.
4. 최종 합격률 계산

```
total_score = min(
    99.9,
    base_score +
    gpa_score +
    eng_score +
    skill_score +
    extra_score
```

	)

3-7. 최종 완성 결과 캡처





3-8. Lessons learned

Plus	혼자서 코드를 짤 때는 내 마음대로 하면 그만이었지만, 프론트엔드와 백엔드를 나누어 협업하면서 서로 데이터를 어떻게 주고받을지 약속하고 소통하는 과정이 재밌었다. 또한 배웠던 알고리즘에 대해서 이번 프로젝트를 통해 내가 직접 구현해보면서 더 깊게 알 수 있었고, 또 정말 그 알고리즘을 이해하고 있는 것이 무엇보다 중요하다는 것을 깨닫게 되었다.
Minus	막혔던 부분에서 AI의 도움을 받았었는데 오히려 AI도 상황을 올바르게 해결하지 못하는 경우가 종종 있었다. 돌아보고 나니 내가 로직을 조금 더 정확하게 이해해서 AI에게 제대로 전달했으면 하는 아쉬움이 남았다.

Interesting	이번 프로젝트를 통해 실제 합격자 스펙이나 기업 요구 기술이 담긴 진짜 데이터를 직접 가공하고 분석해 볼 수 있어서 매우 흥미로웠다. 또한 미흡하지만 진짜 작동하는 서비스를 구현해 보며 성취감을 느꼈다.

4. 졸업요건 진단 시스템(가칭) - 202235330 정회훈

기능명	졸업요건 진단 시스템	담당자	202235330 정회훈
4-4. Input 데이터			
- subjects.csv: 컴퓨터공학과가 이수해야하거나 이수할 수 있는 전공,교양과목 데이터 - completed_codes: 사용자가 이미 이수한 과목 코드 리스트 - gen_ed_credits: 기존에 취득한 일반교양 누적 학점 - current_semester: 사용자의 현재 이수 학기 수			
4-5. 사용한 자료구조 & 알고리즘			
자료구조			
리스트 (List)	- 위치: grad_db, valid_completed, sem_c 변수 - 용도: 전체 과목 목록, 이수 완료 과목, 특정 학기에 추천할 과목들을 순차적으로 담아두는 동적 배열로 사용. - 사용 이유: 순서가 중요하거나 데이터를 순회하며 조건에 맞는 값을 동적으로 필터링할 때 가장 직관적이고 효율적이기 때문.		
해시 테이블 (Dictionary)	- 위치: reqs, earned, grad_cat 변수 - 용도: 각 이수구분별(전필, 전선, 교양 등) 요구 학점과 취득 학점을 매핑하여 누적합을 계산. - 사용 이유: Key를 통한 Value 탐색이 O(1)의 시간 복잡도를 가져, 카테고리별 학점을 즉각적으로 연산하고 비교하는 데 최적화되어 있음.		
해시 집합 (Set)	- 위치: seen_groups 변수 - 용도: 과목 코드의 앞 5자리를 저장하여 동일 학수 과목의 중복 수강 및 중복 배정을 방지하는 락(Lock) 주머니로 사용. - 사용 이유: 순차 리스트 탐색 대신 내부 해시 테이블을 통해 멤버십 검사를 O(1)에 끝내어, 실시간 시뮬레이션 속도를 향상시키기 위해 선택.		
알고리즘			
병합 정렬 (Merge Sort)	- 위치: merge_sort_subjects, merge_subjects 함수 - 용도: 시뮬레이션 과정에서 추천할 과목들을 학년 및 이수구분 우선순위 점수를 기준으로 정렬. - 사용 이유: 최악의 경우에도 O(N log N)의 성능을 보장할 뿐만 아니라, 안정 정렬(Stable Sort) 특성을 통해 동일 점수를 가진 과목 간의 원래 순서가 뒤섞이지 않게 통제하기 위해 직접 구현.		
무작위성이 융합된 탐욕 알고리즘 (Randomized	- 위치: simulate_grad 함수 내부의 while 루프, shuffle_within_priority_groups 함수(랜덤 요소) - 용도: 병합 정렬된 과목들을 동일 우선순위 그룹으로 묶고, 그룹 내부에서만 무작위(Shuffle)로 섞은 뒤, 남은 학기 동안 당장 조건에 맞는 최적의 과목들을 골라		

G r e e d y Algorithm)	학기당 최대 제한 학점(18학점)까지 빽빽하게 채워 넣음. - 사용 이유: 전체 시간표 조합을 전수 조사하는 대신, 현재 시점에서 가장 우선순위가 높은 과목을 먼저 채우는 방식이 실제 학생들의 수강신청 시나리오와 가장 부합(Heuristic)하기 때문.
4-6. 설계/구현 방법	
Architecture	[데이터 레이어] subjects.csv (과목 마스터 DB) ➔ Pandas 로드 & 예외 처리 ➔ GRAD_DB (Subject 객체 리스트) ↓ [API / 컨트롤 레이어] 프론트엔드 POST 요청 (/api/grad/simulate) ➔ 기이수 코드, 일반교양 학점, 현재 학기 접수 ↓ [서비스 레이어 (시뮬레이션 엔진)] ① 학수번호 기반 중복 검사 (seen_groups Set 활용, 중복 과목 가감 없이 제외 및 경고 축적) ② 영역별(전필/전선/교양 3종) 필수 학점 및 총학점(120학점) 진단 ➔ 충족 시 축하 문구 반환 ③ 미충족 시 시뮬레이션 루프 가동 (최대 10학기 한도) ├── 학기 문맥 분석 (홀/짝수 학기 여부 판별 및 현재 가상 학년 계산) ├── 동적 제약조건 필터링 후 merge sort로 정렬 (기이수 제외, 개설 학기 일치 여부, 학년 월반 제한 검사) ├── 우선순위 산정 (추천학년 및 이수구분 점수 결합) 및 동일 그룹 내 부분 무작위 셔플 └── 그리디 알고리즘으로 배정 (학기당 최대 18학점 제약 조건 하에 교양 선점 후 순차 적재) ↓ [출력 레이어] 학기별 추천 과목 및 누적 학점 지표를 포함한 최종 정밀 진단 텍스트(리포트) 생성 ➔ JSON 반환
동작 방식	1. 학수 데이터 정규화 및 기이수 진단 (초기화 단계) 시스템이 구동되면 외부 CSV 파일에서 전공 및 교양 과목 마스터 데이터를 읽어와 예외 처리(BOM 제거, 공백 제거, cp949/utf-8 인코딩 대응)를 거친 후 고유 객체 배열로 상주. 사용자가 입력한 기이수 과목 코드를 8자리 문자열로 정규화(zfill(8))하여 동치 비교의 정확성을 확보. 학수번호의 앞 5자리를 추출하여 동일 그룹(유사/대체 과목) 중복 수강 여부를 해시 집합(seen_groups)을 통해 O(1) 속도로 전수 검사하고, 중복 과목은 진단에서 제외. 2. 제약 조건 기반의 동적 후보군 필터링 (루프 진입 단계) 총학점이 120학점 미만이거나 졸업요건이 미달할 경우 학기별 시뮬레이션을 가동. 매 가상 학기 루프마다 현재 학기 수에 맞춰 '홀수(1학기)/짝수(2학기)' 개설 여부를

	판별하고, 가상 학년을 계산하여 현재 학년보다 높은 고학년 전공 과목이 추천되는 '학년 월반'을 논리적으로 차단. 3. 학년-이수구분 융합 정렬 및 무작위성 부여 (스케줄링 단계) 추출된 가용 과목들을 대상으로 정렬을 수행. 최우선 기준인 '추천 학년'과 차순위 기준인 '이수 구분(전필 전선 계열 융합 기초 순)'을 결합하여 복합 우선순위 점수를 연산. 이때 지정 학년이 없는 '공통 교양' 과목은 현재 가상 학년 점수를 동적으로 부여함으로써, 다음 학년의 전공 과목보다 항상 먼저 수강할 수 있도록 유연성을 부여. 무작위성을 결합한 안정 정렬 알고리즘을 통해 점수가 동일한 구간 내에서만 과목들을 무작위로 뒤섞어(Shuffle), 학년 연동 제약은 사수하면서도 시뮬레이션 시마다 다양한 교양을 추천하는 다양성 확보 4. 학기 제약 한도 내 탐욕적 배정 (최적화 단계) 한 학기 최대 수강 가능 학점인 18학점이 상한선(Constraint). 학생들의 현실적인 수강 패턴을 반영하여, 매 학기 정렬된 후보 중 '교양 과목'을 최소 1개 강제 선점하여 배정함으로써 전공 과목만 한 학기에 몰아서 듣게 되는 현상 방지. 이후 우선순위가 높은 남은 전공 및 필수 과목들을 제한 학점 범위 내에서 탐욕적으로 최대한 가득 채워 담고, 과목이 담길 때마다 카테고리별 득점 상황과 누적 총 학점을 실시간 갱신하여 다음 배정에 즉각 반영. 최종 졸업 요건이 완전히 충족되거나 최대 연산 한계인 10학기에 도달하면 루프를 탈출하고 가독성 높은 리포트 문자열을 빌드하여 프론트엔드로 전송.
핵심 코드	1) Merge sort 및 과목별 우선순위 과정

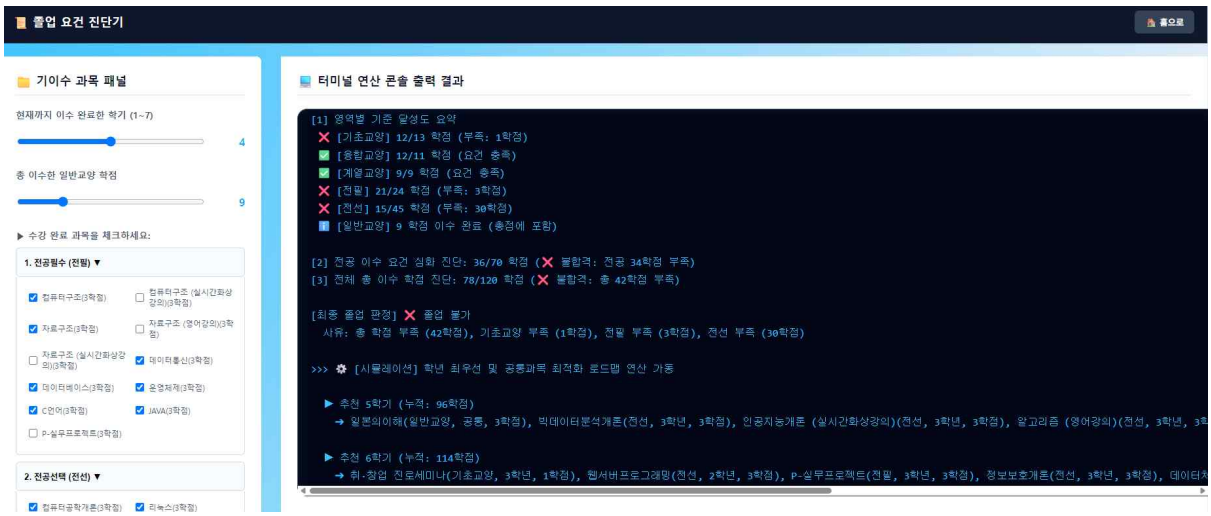
```python
def subject_priority(sub, current_grade):
    1. 주전학년 (최우선): 1학년(100점) < 2학년(200점) < 3학년(300점) < 4학년(400점)
        💡 학년이 지정되지 않은 '공통' 과목은 현재 학년 점수(current_grade * 100)를 부여하여
        다음 학년 전공과목((current_grade + 1) * 100)보다 항상 우선순위가 높도록 설계합니다.

    2. 이수구분 (차순위): 전필(0점) < 전선(1점) < 계열교양(2점) < 융합교양(3점) < 기초교양(4점) < 일반교양(5점)
    """
    try:
        grade_score = int(sub.recommended_grade) * 100
    except ValueError:
        grade_score = current_grade * 100

    if sub.category == "전필":
        cat_score = 0
    elif sub.category == "전선":
        cat_score = 1
    elif sub.category == "계열교양":
        cat_score = 2
    elif sub.category == "기초교양":
        cat_score = 3
    elif sub.category == "융합교양":
        cat_score = 4
    else:
        cat_score = 5

    return grade_score + cat_score

def merge_subjects(left, right, current_grade):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if subject_priority(left[i], current_grade) <= subject_priority(right[j], current_grade):
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result

def merge_sort_subjects(arr, current_grade):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort_subjects(arr[:mid], current_grade)
    right = merge_sort_subjects(arr[mid:], current_grade)
    return merge_subjects(left, right, current_grade)
```

2) 랜덤요소에 기반한 그리디

```python
def shuffle_within_priority_groups(arr, current_grade):
    """
    동일 우선순위(점수) 그룹 내에서만 shuffle — 학년 정렬은 유지
    ex) 1학년 기초교양 5개끼리는 랜덤, 2학년 전필이 앞으로 튀어나오는 일은 없음
    """
    if not arr:
        return arr
    sorted_arr = merge_sort_subjects(arr, current_grade)
    result = []
    for _, group in groupby(sorted_arr, key=lambda s: subject_priority(s, current_grade)):
        group_list = list(group)
        random.shuffle(group_list)
        result.extend(group_list)
    return result
```

```python
while (cur_tot < 120 or not all(earned[cat] >= reqs[cat] for cat in reqs)) and target_sem <= 10:
    sem_type = "짝수" if target_sem % 2 == 0 else "홀수"
    current_grade = min((target_sem + 1) // 2, 4)

    avail = [s for s in GRAD_DB
             if s.code[:5] not in seen_groups
             and needs(s.category, cur_tot)
             and (s.semester_type == "모든학기" or sem_type in s.semester_type)
             and (not s.recommended_grade.isdigit() or int(s.recommended_grade) <= current_grade)]

    if not avail:
        break

    sem_cred = 0
    sem_c = []

    # 교양 강제 선점: 학년 우선순위 유지 + 동점 그룹 내 랜덤
    ge_subs = [s for s in avail if "교양" in s.category]
    if ge_subs:
        ge_subs = shuffle_within_priority_groups(ge_subs, current_grade)
        chosen_ge = ge_subs[0]

        sem_c.append(chosen_ge)
        sem_cred += chosen_ge.credits
        seen_groups.add(chosen_ge.code[:5])
        earned[chosen_ge.category] += chosen_ge.credits
        cur_tot += chosen_ge.credits

        avail = [s for s in avail if s.code[:5] != chosen_ge.code[:5]]

    # 교양 선점 후 cur_tot 갱신 반영하여 재필터 + 학년 우선순위 유지 + 동점 그룹 내 랜덤
    avail = [s for s in avail if needs(s.category, cur_tot)]
    selected_avail = shuffle_within_priority_groups(avail, current_grade)

    for s in selected_avail:
        if s.code[:5] in [c.code[:5] for c in sem_c]:
            continue

        if sem_cred + s.credits <= 18:
            sem_c.append(s)
            sem_cred += s.credits
            seen_groups.add(s.code[:5])
            earned[s.category] += s.credits
            cur_tot += s.credits
```

4-7. 최종 완성 결과 캡처



4-8. Lessons learned

Plus	수업시간에 배운 알고리즘을 실사용함으로서 구체적 이해 가능, 팀 프로젝트 경험을 쌓아 나와는 다른 방식으로 코드를 짠 사람과의 교류를 통한 시야의 발전, ai를

	통한 바이브 코딩으로 ai 활용 능력 증가
Minus	중간중간 기존에 했던 것보다 더 좋은 아이디어가 떠올라 잦은 수정이 일어나고 그로 인해 팀원 간 일정 조율의 어려움, 세부적인 사항의 미구현이나 기능에 대한 아쉬움(학과별, 학번별 구현이나 1학기를 정상적으로 이수했다고 가정해야만 제대로 돌아가는 코드 등),개발 과정에서 예상보다 더 많은 시간이 소요됨
Interesting	사용자 경험에 있어 개발을 하는 사람마다 다양한 관점을 가지고 있어 시각차가 있다는 것을 알게 됨, 다른 사람의 코드를 보고 상호적으로 소통하며 자신의 코드도 다시 한 번 돌아볼 수 있다는 점을 다시금 깨달음, 데이터를 가공하는 과정이 내가 기존에 생각했던 것보다 복잡한 과정이라는 사실을 알게 됨